Master's Degree Programme — Graduate Course

Thermodynamic Computing for Artificial Intelligence

Physics, Probabilistic Hardware, Algorithms and Thermodynamic Language Models

Methodological Plan & Syllabus (Pensum)

Topics · Learning outcomes · Methodology · Coding labs · Projects
Assessment · Video references · Bibliography

Level:	Master's (MSc / MEng), elective or specialisation course
Duration:	16 weeks · 64 contact hours · 128 independent hours
Credits:	4 academic credits (1 credit = 48 hours of total student work)
Modality:	Theoretical–practical (lecture + computational lab), on-site or hybrid
Version:	1.0 — October 2026
Instructor:	_____
Institution:	_____

Companion notes: `thermodinamic.md` (physics, hardware, energy, Shor analogy) and `thermodinamic-llm.md` (Z1T, diffusion transformers, TNGD, case-study pipeline).

Contents

1 Course Identification **2**

2 Description and Justification **2**

3 Objectives **3**
3.1 General objective . . . 3
3.2 Specific objectives . . . 3

4 Learning Outcomes (Competencies) **3**

5 Methodology **4**
5.1 Pedagogical approach . . . 4
5.2 Weekly learning cycle . . . 5
5.3 Didactic principles and rules of practice . . . 5
5.4 Learning environment and tools . . . 5
5.5 Levelling and inclusion . . . 5

6 Pensum: Complete List of Topics **6**

7 Weekly Schedule **11**

8 Coding Tasks: Laboratory Programme **11**

9 Capstone Research Projects **16**

10 Assessment Plan **17**
10.1 Laboratory rubric . . . 17
10.2 Capstone rubric . . . 17

11 Video References **18**

12 Journal Club Reading List **19**

13 Bibliography **19**

A Prerequisite Self-Check (Week 1) **21**

B Formula Sheet **22**

C Sample Exam Questions **22**

D Lab Environment Setup **23**

1 Course Identification

Course name	Thermodynamic Computing for Artificial Intelligence
Suggested code	TC-7xx (to be assigned by the programme)
Programme	MSc in Artificial Intelligence / Physics / Electronic Engineering / Applied Mathematics
Course type	Elective, research-oriented; theoretical–practical
Credits and workload	4 credits = 192 h: 64 h contact (2 h lecture + 2 h lab per week) + 128 h independent work (about 8 h per week)
Duration	16 weeks (15 teaching weeks + 1 week of capstone presentations)
Prerequisites	Linear algebra, probability and statistics, calculus and ODEs, Python programming
Recommended	Introductory machine learning / deep learning; introductory statistical mechanics; basic circuits and signals
Target students	Graduate students from computer science, AI, physics, electrical/-electronic engineering and applied mathematics
Languages and tools	Python, JAX/Equinox/Optax, NumPy/SciPy, THRML, Torx, ngspice, Git, LaTeX

2 Description and Justification

Artificial intelligence is running into an *energy wall*. Grid capacity is now a main obstacle to building data centres, and most of the energy in a digital accelerator is spent *moving* bits and *suppressing* thermal noise, not computing. Meanwhile, the dominant AI workloads (diffusion models, token sampling, Bayesian inference, stochastic optimisation) are inherently *probabilistic*: they deliberately inject randomness that digital hardware must synthesise at great cost.

Thermodynamic computing reverses this logic. It uses thermal fluctuations and the natural relaxation of physical systems towards equilibrium as computational resources. A *thermodynamic sampling unit* (TSU) receives the parameters of an energy function and returns samples from the corresponding Boltzmann distribution. Examples are probabilistic bits (pbits) on an Ising graph, and coupled oscillators that solve linear systems by equilibrating. Extropic's Z1 chip (269,568 pbits, early access planned for 2027), Normal Computing's stochastic processing units, magnetic p-bit computers and Brownian token circuits show the field moving from theory to silicon. On the algorithm side there are thermodynamic linear algebra, thermodynamic natural gradient descent (TNGD), denoising thermodynamic models (DTM) and Z1T sparse transformer-like language models.

The field lacks people who can move fluently across *statistical physics*, *hardware constraints* and *modern machine learning*. This course trains them. It is built around a reproducible, code-first methodology. Students derive each principle, simulate it, benchmark it against a digital baseline, and audit every energy claim with an explicit cost model. They also learn to label each figure as *measured*, *simulated*, *projected*, *proved* or *estimated*.

Central question of the course

When, why and by how much can physics-based probabilistic hardware make AI more energy-efficient, and what must change in our algorithms for that to happen?

3 Objectives

3.1 General objective

Train students to understand, design, simulate and critically evaluate thermodynamic and probabilistic computing systems for artificial intelligence, from the physical principles to hardware-aware algorithms and thermodynamic language models.

3.2 Specific objectives

SO1. Master the physical foundations: Boltzmann statistics, Langevin and Fokker–Planck dynamics, Landauer's principle, fluctuation theorems, speed limits and uncertainty relations.

SO2. Formulate computational problems as energy-based programs (Ising models, quadratic potentials, Boltzmann machines) and solve them by sampling or equilibration.

SO3. Analyse the architecture and the constraints of thermodynamic hardware: fixed connectivity, precision, mismatch, I/O and digital glue.

SO4. Implement thermodynamic algorithms for linear algebra, optimisation (TNGD, K-FAC) and number theory (diffusion order finding), and compare them with digital and quantum approaches.

SO5. Design hardware-aware generative models: denoising thermodynamic models, masked diffusion language models and Z1T-style sparse transformers.

SO6. Build and use explicit energy models and metrics (J/token, J/FLOP, MFU, entropy production, efficiency η, I/O share) to evaluate claims rigorously.

SO7. Carry out a reproducible research project and communicate it in a scientific report and an oral presentation, using AI research assistants responsibly.

4 Learning Outcomes (Competencies)

Code	By the end of the course the student is able to…	Bloom level	Evidence
LO1	Explain the physical principles of thermodynamic computing (Boltzmann distribution, Langevin dynamics, Landauer bound, Jarzynski/Crooks, TUR, Wasserstein speed limit).	Understand	Quizzes, midterm
LO2	Derive and simulate stochastic dynamics (OU, Langevin, Gibbs, master equations) and verify stationary distributions, mixing and ergodic averages.	Apply / Analyse	Labs 2–5
LO3	Formulate ML and linear-algebra problems as energy-based programs and map them onto hardware graphs.	Apply / Create	Labs 5–8

Code	By the end of the course the student is able to…	Bloom level	Evidence
LO4	Analyse hardware constraints (degree, precision, mismatch, I/O, digital glue) and their effect on algorithms.	Analyse	Lab 7, Lab 14, seminar
LO5	Implement and benchmark thermodynamic algorithms (linear solvers, TNGD, thermodynamic K-FAC, diffusion order finding) against digital and quantum baselines.	Apply / Evaluate	Labs 8–11
LO6	Design hardware-aware generative models (DTM, masked diffusion LMs, Z1T-style transformers).	Create	Labs 12–14, capstone
LO7	Evaluate energy claims with explicit cost models and metrics, and distinguish measured, simulated, projected and estimated evidence.	Evaluate	All labs, capstone
LO8	Reproduce and extend a research result; write and present it professionally; use AI agents with disclosure and verification.	Create	Journal club, Lab 15, capstone

5 Methodology

5.1 Pedagogical approach

The course follows a **spiral, research-based** model, *Physics* → *Program* → *Hardware* → *Model*. It revisits the same objects (the Boltzmann distribution, the OU process, the heat kernel, the pbit) at increasing levels of abstraction. Five strategies are combined:

M1. Flipped classroom. A curated video (Section 11) and a short reading come before each session, with 3 guiding questions that are answered in class.

M2. Active lectures. Board derivations alternate with concept checks (peer instruction) and short "energy-accounting" exercises.

M3. Computational labs (problem-based learning). Weekly pair-programming labs in JAX notebooks. Every lab ends with a digital baseline comparison and an energy or complexity table.

M4. Journal club. Each student presents one paper (15 minutes) and leads the discussion. The aim is critical reading of claims and of the evidence labels behind them.

M5. Project-based learning. A capstone research project following the cycle *Reproduce* → *Ablate* → *Extend*. This is the same cycle that research agents are trained on in Unit 11.

5.2 Weekly learning cycle

Moment	Time	Activities
Before class	2–3 h	Watch the video of the week; read the assigned paper section; answer 3 guiding questions in the course forum.
Session A (lecture)	2 h	10 min recap quiz; 60 min derivations and concepts; 20 min concept checks; 15 min journal club; 15 min lab briefing.
Session B (lab)	2 h	Guided implementation in pairs; checkpoints validated by the instructor/TA; starter notebooks with tests.
After class	4–5 h	Finish the lab notebook and report (plots, tables, evidence labels); work on the capstone.

5.3 Didactic principles and rules of practice

- **Simulation before formalism.** Students see the phenomenon (a relaxing distribution, a mixing chain) before proving the theorem.
- **Always a baseline.** Every thermodynamic algorithm is compared with the best digital method (CG, Cholesky, Adam, GPU inference) and, where relevant, with the quantum analogue.
- **Energy accounting.** Each result reports $E_{\text{total}} = E_{\text{fabric}} + E_{\text{I/O}} + E_{\text{digital}}$ and states which terms are measured or assumed.
- **Evidence labels.** Each number carries one of: [M] measured on hardware, [S] simulated, [P] vendor projection, [T] theorem, [E] own estimate.
- **Reproducibility.** Seeds fixed, environments pinned, results regenerated with one command, and a short `README` for each lab.
- **Responsible AI assistance.** Coding assistants and LLMs are allowed with disclosure. Students must verify every derivation and number they submit.

5.4 Learning environment and tools

- **Languages and libraries:** Python 3.11+, JAX, Equinox, Optax, NumPy/SciPy, Matplotlib, NetworkX, SymPy.
- **Thermodynamic stack:** THRML (`https://docs.thrml.ai/en/latest/`), Torx (`https://docs.torx.ai`, `https://github.com/extropic-ai/torx`), the Extropic simulator API (early access, `https://extropic.dev`), and `posteriors` (Normal Computing, `https://github.com/normal-computing/posteriors`).
- **Circuits:** ngspice for RC/oscillator networks.
- **Measurement:** NVIDIA NVML (`nvidia-ml-py`) to log GPU power and compute J/token. JSONL metric logs.
- **Compute:** CPU is enough for Units 0–7. One consumer GPU (8 GB or more) or a cloud notebook is needed for Units 8–11.
- **Collaboration:** Git repositories per team; reports in LaTeX (IEEE two-column template).

5.5 Levelling and inclusion

Two levelling notebooks are given in Week 1: *Physics for computer scientists* (ensembles, free energy, SDEs) and *ML for physicists* (autodiff, optimisers, transformers). Office hours and a teaching

assistant support students with uneven backgrounds. Labs have a *core* part and a *challenge* part.

6 Pensum: Complete List of Topics

The course is organised in 12 units (0–11). Each unit lists the topics students must master and two guiding questions.

Unit 0 — Orientation and Mathematical Levelling (Week 1)

- 0.1 The AI energy wall: data-centre power, end of Dennard scaling, communication energy ($\propto CV^2$), von Neumann bottleneck.
- 0.2 Map of computing paradigms: classical, thermodynamic, quantum, neuromorphic, analog, reversible; the three fluctuation domains.
- 0.3 Probability refresher: Bernoulli, categorical and Gaussian variables; multivariate Gaussians; moments; Shannon entropy; KL divergence.
- 0.4 Linear algebra refresher: SPD matrices, eigendecomposition, condition number κ, Kronecker products, matrix exponential, Woodbury identity.
- 0.5 Markov chains: transition matrices, stationary distributions, detailed balance, spectral gap, mixing time.
- 0.6 Fourier analysis on finite abelian groups: characters of $\mathbb{Z}/r\mathbb{Z}$, orthogonality.
- 0.7 JAX essentials: pure functions, PRNG keys, `vmap`, `jit`, `scan`, `jvp`/`vjp`.

Guiding questions: Where does the energy of a GPU actually go? Why would randomness be a resource rather than a nuisance?

Unit 1 — Thermodynamics and Information (Week 2)

- 1.1 Laws of thermodynamics; equilibrium; internal energy, entropy, free energy $F = U - TS$.
- 1.2 Statistical mechanics: microstates, canonical ensemble, Boltzmann distribution $p(x) = e^{-\beta E(x)}/Z$, partition function.
- 1.3 Shannon versus Gibbs entropy; information is physical.
- 1.4 Maxwell's demon and the Szilard engine.
- 1.5 Landauer's principle ($k_B T \ln 2 \approx 2.87 \times 10^{-21}$ J at 300 K); logical versus thermodynamic reversibility; Bennett's reversible computing.
- 1.6 Thermal noise: Johnson–Nyquist and $k_B T/C$ noise; fluctuation scales of devices.
- 1.7 Historical arc: Carnot, Boltzmann, Gibbs, Shannon, Landauer, Hopfield, Hinton, Jarzynski, Crooks.

Guiding questions: What does it cost to erase a bit, and when is that cost paid? Why do digital computers operate millions of times above the Landauer limit?

Unit 2 — Stochastic Dynamics (Week 3)

- 2.1 Brownian motion, the Wiener process, essentials of Itô calculus.
- 2.2 Langevin equation, overdamped and underdamped: $\dot{x} = \mu f(x) + \sqrt{2\mu k_B T}\,\xi(t)$; mobility and device timescales.

2.3 Fokker–Planck equation, probability currents, stationary solutions.

2.4 The Ornstein–Uhlenbeck process: closed-form solution; $x_\infty \sim \mathcal{N}(A^{-1}b, \beta^{-1}A^{-1})$.

2.5 Fluctuation–dissipation theorem.

2.6 Ergodicity: time versus ensemble averages, autocorrelation, integration time, error bars.

2.7 Numerical SDEs: Euler–Maruyama, stability ($\Delta t < 2/\lambda_{\max}$), higher-order schemes.

2.8 Continuous-time Markov jump processes: master equation, Gillespie algorithm, first-passage times.

Guiding questions: How can the *mean* of a noisy process solve a deterministic equation? What limits how fast it converges?

Unit 3 — Stochastic Thermodynamics (Week 4)

3.1 Trajectory-level heat, work and entropy (Sekimoto, Seifert).

3.2 Entropy production $\Sigma \geq 0$ and the second law for single trajectories.

3.3 Fluctuation theorems: Crooks, Jarzynski $\langle e^{-\beta W} \rangle = e^{-\beta \Delta F}$; free-energy estimators and their bias.

3.4 Thermodynamic speed limits: $\tau \geq W_2^2/(\mu k_B T \Sigma)$; Wasserstein distance, optimal transport, Gelbrich bound; efficiency $\eta \leq 1$.

3.5 Protocol design: gradual loading, clock acceleration (heat $\times\lambda$, power $\times\lambda^2$, same Σ).

3.6 Thermodynamic uncertainty relations (kinetic and mixed bounds) for computation time.

3.7 Thermodynamics of resetting and of error-free readout.

3.8 Thermodynamic length and energy–time trade-offs.

3.9 Non-equilibrium conductance matrices and circuit theory of chemical reaction networks (resistance addition).

Guiding questions: Can a computer be fast *and* efficient? What exactly do we pay for when we speed it up?

Unit 4 — Energy-Based Models, Ising Models and Neural Networks (Weeks 5–6)

4.1 Energy-based models and probabilistic graphical models; factor graphs and hypergraphs.

4.2 The Ising model $E(s) = -\sum_{i<j} J_{ij} s_i s_j - \sum_i h_i s_i$; ferromagnets, spin glasses, phase transitions (intuition).

4.3 MCMC: Metropolis–Hastings, Gibbs sampling, block and *chromatic* Gibbs, graph colouring.

4.4 Diagnostics: burn-in, autocorrelation time, effective sample size; simulated annealing; parallel tempering.

4.5 The pbit as a stochastic neuron: $P(s_i{=}{+}1) = \sigma(2\beta u_i)$, $\mathbb{E}[s_i] = \tanh(\beta u_i)$.

4.6 Hopfield networks and associative memory (the $T \to 0$ Ising model).

4.7 Boltzmann machines and their learning rule $\Delta J_{ij} \propto \langle s_i s_j \rangle_{\text{data}} - \langle s_i s_j \rangle_{\text{model}}$.

4.8 RBMs, contrastive divergence (CD-k), persistent CD; deep belief nets; wake–sleep and Helmholtz machines.

4.9 Mean-field theory: from sampling to deterministic tanh layers.

4.10 Ising formulations of optimisation (Max-Cut, SAT) and invertible probabilistic logic (factorisation by clamping a multiplier).

Guiding questions: In what precise sense is a pbit a neuron? Why did the field abandon Boltzmann machines, and why might hardware bring them back?

Unit 5 — Probabilistic and Thermodynamic Hardware (Week 7)

5.1 Why digital hardware is inefficient for probabilistic AI: data movement, memory hierarchy, PRNG cost, noise margins.

5.2 p-bits: magnetic-tunnel-junction and CMOS implementations; p-computers; sparse Ising machines.

5.3 Extropic TSUs: X0 and XTR-0; Z1 (269,568 pbits, 16 couplings per pbit, more than 50 MHz, under 1 W); M.2, PCIe and billion-pbit cluster systems; primitives pbit, pdit, pmode, pmog.

5.4 Continuous stochastic processing units (Normal Computing): coupled RC/LC oscillators; 8-cell PCB prototype; first-chip tape-out.

5.5 Other substrates: Brownian token circuits (Hub, CJoin, Ratchet; skyrmions, single electrons); Josephson and mechanical Langevin devices; RC diffusion networks; chemical reaction networks.

5.6 Analog non-idealities: component mismatch, input/output quantisation, nonlinearity, shot noise; error mitigation (stochastic rounding, conservative quantisers).

5.7 Hardware abstractions: fixed parent graph, embedding programs into degree-limited graphs, the N^2 connectivity problem.

5.8 Energy accounting in heterogeneous systems (TSU + GPU/XPU + FPGA): fabric versus I/O versus digital glue.

Guiding questions: If a pbit update costs tens of femtojoules, why does a full system still spend most of its energy elsewhere? What does a fixed degree of 16 imply for algorithms?

Unit 6 — Thermodynamic Algorithms I: Linear Algebra and Inference (Week 8)

6.1 Linear systems from the first moment of an OU process.

6.2 Matrix inverse and Lyapunov equations from second moments.

6.3 Determinants from Jarzynski work statistics ("all-moment" algorithms); the moment hierarchy.

6.4 Complexity: $O(d\kappa^2\varepsilon^{-2})$ versus $O(\min\{d^\omega, d^2\sqrt{\kappa}\})$; overdamped versus underdamped regimes; the energy–time bound.

6.5 Timing models: DAC/ADC, compilation cost $O(d^2)$, RC time constants; why compilation often dominates.

6.6 Survey: thermodynamic Bayesian inference, quadratic programming, matrix exponentials and thermodynamic parallelism.

Guiding questions: Where does the linear-in-d speed-up come from? Which assumptions fail first in a real device?

Unit 7 — Thermodynamic Algorithms II: Diffusion Computing and the Shor Analogy (Week 9)

7.1 Reduction of factoring to order finding; Chinese remainder theorem; success probability $1-(m+1)/2^m$.

7.2 Shor's algorithm: modular exponentiation, quantum Fourier transform, period finding, continued fractions.

7.3 Random walks on Cayley graphs; the discrete-time heat kernel; spectral expansion through characters.

7.4 The diffusion order-finding theorem: half-lazy walk with dyadic generators $b^{\pm 2^t}$; doubling lemma; lacunary exponential sums; $n_0 = 4(M+1)(\log_2 N + 2)$; $r = \mathrm{round}(1/p_n(e))$.

7.5 Trigonometric sum identities from the uniqueness of the heat kernel (combinatorial versus spectral form).

7.6 Physical realisation with RC networks ($C\dot{V} = -LV$); resource analysis: $\Omega(r)$ nodes and $\sim 2\log_2 N$ bits of analog precision.

7.7 Collision (relation-finding) variant: birthday bound $T \approx \sqrt{r}$, gcd stabilisation and $1/\zeta(s)$; link with Pollard's rho.

7.8 Comparative complexity: quantum steps versus diffusion steps versus digital steps; HHL versus thermodynamic linear algebra.

Guiding questions: Shor amplifies Fourier peaks with interference, while diffusion damps all non-zero Fourier modes. What does each one cost in physical resources?

Unit 8 — Thermodynamic Machine Learning: Training (Weeks 10–11)

8.1 Information geometry: Fisher information, KL geometry, natural gradient.

8.2 Curvature matrices: empirical Fisher, generalised Gauss–Newton (GGN); NGD-CG; NGD-Woodbury.

8.3 Thermodynamic natural gradient descent (TNGD): OU dynamics, analog time interpolating SGD ($t = 0$) and NGD ($t \to \infty$), delay as momentum, TNGD-Adam, noise robustness, hardware (resistor arrays).

8.4 K-FAC: Kronecker factors, EMA, the expand and reduce variants for weight sharing.

8.5 Thermodynamic K-FAC: per-layer $n \times n$ solves, complexity $O(bn^2 + n^2\kappa^2)$, input/output quantisation (at least 8-bit output).

8.6 Non-linear Langevin computers trained as classifiers; training for a readout time; computing out of equilibrium; efficiency of inference protocols.

8.7 Hardware-aware training: noise injection, 4-bit quantisation-aware training, fixed sparsity from step 0.

Guiding questions: Why are second-order methods rarely used on GPUs? Which parts of TNGD remain digital, and do they dominate?

Unit 9 — Generative Models on Thermodynamic Hardware (Week 12)

9.1 Diffusion models: DDPM, score matching, the SDE view, probability-flow ODE, samplers and step counts.

9.2 Discrete diffusion: absorbing-state (masked) diffusion; the ELBO (MDLM); few-step

samplers and distillation.

9.3 Denoising thermodynamic models (DTM): chains of conditional EBMs sampled by Gibbs.

9.4 Cost model: energy per token $\approx T \cdot e_{\text{pos}}$; GPU utilisation (MFU) versus spatial TSUs; precision schedules across denoising steps.

9.5 Efficiency analysis: Route A (porting a DiT or masked-diffusion transformer) versus Route B (native thermodynamic denoising).

Guiding questions: Why is diffusion the most "thermodynamic-native" generative paradigm? When does the number of denoising steps erase the hardware advantage?

Unit 10 — Thermodynamic Language Models (Weeks 13–14)

10.1 Transformer review: attention, MLP, normalisation, KV cache, prefill versus decode, MFU.

10.2 Attention-free and linear attention: AFT, AFT-conv, gated convolutional attention (GCA).

10.3 DyT normalisation $\gamma \tanh(\alpha x) + \beta$ and its mapping to tanh units.

10.4 Sparse transformers: fixed connectivity versus sparsity percentage; scaling laws with a connectivity axis; iso-FLOP analysis; compute-optimal training.

10.5 Encoding numbers in pbits: dy4p / probabilistic integers; effective bits $\approx b_1 + \frac{1}{2}\log_2 S$; precision on demand.

10.6 Tanh-linear units; embedding projections into a degree-16 graph (4 inputs $\times$ 4 pbits).

10.7 The Z1T architecture and disaggregated inference (Z1 + FPGA/XPU); energy and latency tables; the FPGA bottleneck; the vocabulary head.

10.8 Energy metrics for LLMs: J/token, J/FLOP, MFU, "headline versus net" (sparsity tax).

10.9 Case study: an AFT-Transformer + TNGD JAX pipeline (autoregressive and masked diffusion) with energy logging and TSU-mapping records.

Guiding questions: Which operations of a transformer are native to a TSU, and which are not? Why do TSUs shine at batch-1 decode but not at prefill?

Unit 11 — Software Stack, Research Agents and Frontiers (Week 15)

11.1 THRML: factor graphs, block Gibbs, simulating TSUs on GPUs.

11.2 Torx: stochastic differentiable programming; stochastic transition kernels; parameterised stochastic circuits.

11.3 Thermalizers: variational compilation of stochastic programs into hardware EBMs; compilation error in deep programs.

11.4 The simulator API and agentic research workflows.

11.5 LLM research agents: GRPO, execution plus rubric rewards, LLM-as-judge, reward hacking; the "Thermo-RSI" results.

11.6 Open problems: connectivity versus quality, removing digital glue, precision, theory of thermodynamic complexity, energy policy and the societal impact of AI energy use.

Guiding questions: Can agents speed-run fifteen years of algorithm co-evolution? What would convince you that a thermodynamic advantage is real?

7 Weekly Schedule

Videos are coded V1–V18 (Section 11); readings refer to the bibliography.

Wk	Unit and main topics	Lab / deliverable	Videos	Readings
1	U0: energy wall, paradigms, probability, linear algebra, Markov chains, JAX	L1 JAX & Markov chains	V2, V6	[6, 21, 7]
2	U1: thermodynamics, Boltzmann, Landauer, Maxwell's demon	L2 Bit erasure	V7	[8, 9, 10]
3	U2: Langevin, Fokker–Planck, OU, ergodicity, Euler–Maruyama	L3 OU linear solver	V12 (SDE part)	[1, 46]
4	U3: stochastic thermodynamics, Jarzynski, speed limits, TUR	L4 Jarzynski & speed limits	V8	[11, 2, 42]
5	U4a: EBMs, Ising, MCMC, chromatic Gibbs	L5 Ising on a hardware graph	V4	[13, 3]
6	U4b: Hopfield, Boltzmann machines, RBMs, CD	L6 BARS-RBM & Hopfield; **capstone proposal**	V9	[14, 16, 15]
7	U5: p-bits, TSUs, SPUs, Brownian circuits, non-idealities	L7 p-bit circuits & hardware constraints	V1, V3, V5	[17, 18, 22, 25, 26]
8	U6: thermodynamic linear algebra, timing models; **midterm exam**	L8 TLA benchmark & error mitigation	V3	[30, 31]
9	U7: order finding, Shor, heat kernel on Cayley graphs, collisions	L9 Diffusion factoring	V11, V10	[32, 33, 34, 35]
10	U8a: Fisher, GGN, natural gradient, TNGD	L10 TNGD on MNIST	—	[36, 37, 40]
11	U8b: K-FAC, thermodynamic K-FAC, Langevin computers	L11 Thermodynamic K-FAC; **progress report**	—	[38, 41, 42]
12	U9: diffusion, discrete diffusion, DTM, cost models	L12 DDPM & DTM-style chain	V12, V13, V18	[44, 45, 47, 48]
13	U10a: transformers, AFT/GCA, DyT, sparse scaling	L13 Masked diffusion LM vs AR	V14, V15, V16, V17	[49, 50, 51, 52]
14	U10b: dy4p, tanh-linear, Z1T, energy metrics, case study	L14 Z1T-ification & energy projection	V1	[23]
15	U11: THRML, Torx, Thermalizers, research agents, frontiers	L15 Mini research-agent benchmark	V1	[54, 55, 24, 57]
16	Capstone presentations, peer review, course synthesis	**Final report + talk**	—	—

8 Coding Tasks: Laboratory Programme

Fifteen labs, one per teaching week. Each lab has a *core* part (required) and a *challenge* part (optional, for bonus). Deliverables are a notebook, a 2–3 page report with plots and an evidence-labelled results table, and a test file. The best 12 of 15 labs count towards the grade.

L1 — JAX Warm-up: Randomness and Markov Chains (Week 1)

Objective. Master functional randomness in JAX and the spectral view of Markov chains.
Core tasks.

1. Generate 10^6 Bernoulli samples with `jax.random` and `vmap`; estimate their entropy; time the sampler.
2. Build a 3-state and a 50-state chain; find the stationary distribution by power iteration and by eigendecomposition.

3. Compute the spectral gap and plot $\|p_n - \pi\|_1$ against n; compare with λ_1^n.

Challenge. Estimate the energy per random sample on your GPU with NVML (J/sample). This is the digital baseline used all semester.
Deliverable. Notebook, convergence plot, J/sample table.

L2 — Landauer's Bound: Erasing a Bit in a Double Well (Week 2)

Objective. Observe the cost of erasure in a Langevin simulation.
Core tasks.

1. Simulate an overdamped particle in $V(x) = ax^4 - bx^2 + cx$ (a one-bit memory).
2. Implement an erasure protocol (lower barrier, tilt, raise barrier) of duration τ; measure the average work.
3. Plot $\langle W \rangle(\tau)$ and show that it approaches $k_B T \ln 2$ for slow protocols; explain the excess dissipation of fast protocols.

Challenge. Simulate a Szilard engine and close the information–work cycle.
Deliverable. Work histogram, $\langle W \rangle$ versus τ, short discussion.

L3 — The Ornstein–Uhlenbeck Linear Solver (Week 3)

Objective. Solve $Ax = b$ and invert A by equilibration.
Core tasks.

1. Implement Euler–Maruyama for $dx = -(Ax - b)\,dt + \sqrt{2\beta^{-1}}\,dW$ with Wishart-distributed SPD matrices ($d = 10$ to 500).
2. Estimate $A^{-1}b$ by time averaging after burn-in, and A^{-1} from the covariance.
3. Verify the stability condition $\Delta t < 2/\lambda_{\max}$; plot error against integration time and against d (linear versus quadratic scaling).
4. Compare with conjugate gradient and Cholesky (wall-clock on CPU).

Challenge. Underdamped dynamics; compare convergence as a function of κ.
Deliverable. Error–time curves, crossing-time versus d plot.

L4 — Jarzynski, Speed Limits and Clock Acceleration (Week 4)

Objective. Measure work, heat and entropy production; test fluctuation theorems and speed limits.
Core tasks.

1. Drag a harmonic trap at finite speed; estimate ΔF with Jarzynski and with Bennett's acceptance ratio; study the bias versus the number of trajectories.
2. Estimate $\ln\det A$ of an SPD matrix from work measurements between two quadratic potentials; compare with `slogdet`.
3. For a Gaussian process compute W_2^2 in closed form and the efficiency $\eta = W_2^2/(\mu\tau k_B T\Sigma)$.
4. Apply clock acceleration ($V \to \lambda V$ plus extra noise); verify unchanged Σ and η, heat $\times\lambda$ and power $\times\lambda^2$.

Challenge. Find the loading time that maximises η for a small non-linear (quartic) network.
Deliverable. Table of ΔF estimators; η versus protocol duration.

L5 — Ising Sampling on a Hardware-like Graph (Week 5)

Objective. Implement chromatic Gibbs sampling, the core operation of a TSU.
Core tasks.

1. Checkerboard (2-colour) Gibbs for the 2D Ising model in JAX; magnetisation and susceptibility against temperature.
2. Measure autocorrelation times near the critical temperature; compute the effective sample size.
3. Build a sparse graph with degree 16 (Z1-like) and colour it greedily; count colours and updates per second.

Challenge. Parallel tempering; compare mixing on a frustrated spin glass.
Deliverable. Phase diagram, autocorrelation plot, throughput table (updates/s, J/update on GPU).

L6 — Learning by Sampling: BARS-RBM and Hopfield Memory (Week 6)

Objective. Reproduce a classic experiment from the Hinton tradition.
Core tasks.

1. BARS dataset: 4×4 images made of random bars; train an RBM with 8 hidden units using CD-1.
2. Visualise receptive fields; each hidden unit should learn one bar.
3. Hopfield network: store P random patterns in N units; measure retrieval against P/N (capacity of about $0.14N$).

Challenge. RBM on binarised MNIST with persistent CD; generate samples by Gibbs.
Deliverable. Receptive-field grid, reconstruction error, capacity curve.

L7 — p-bit Circuits and Hardware Constraints (Week 7)

Objective. Program a probabilistic computer and feel the hardware limits.
Core tasks.

1. Implement p-bits with sigmoid activation and an invertible AND gate (run forwards and backwards).
2. Write the Ising form of a small binary multiplier; clamp the output to 35 or 143 and sample the inputs (factorisation by sampling).
3. Embed a dense 32-node problem into a degree-16 graph (chains or sparsification); compare solution quality.
4. Add coupling mismatch (σ from 1% to 20%) and quantise weights to 4 bits; measure the degradation.
5. Estimate energy with $E = n \cdot n_{\text{sweeps}} \cdot e_{\text{update}} + b_{\text{I/O}} \cdot e_{\text{bit}}$ (Z1-like $e_{\text{update}} \leq 7.4 \times 10^{-14}$ J; sweep e_{bit}).

Challenge. Gillespie simulation of a Brownian token half-adder; measure the distribution of first-passage times and check the TUR.
Deliverable. Success probability against sweeps, mismatch study, energy table with I/O share.

L8 — Thermodynamic Linear Algebra Benchmark (Week 8)

Objective. Benchmark honestly, including digital overheads.
Core tasks.

1. Build a timing model (16-bit DAC/ADC, 50 Gb/s link, $RC = 1\,\mu$s, $O(d^2)$ compilation) and estimate end-to-end time.
2. Compare with CG and Cholesky for $d \in \{100, 1000, 5000\}$ and several κ; identify the dominating term.
3. Implement a conservative (diagonally dominant) quantiser and stochastic rounding of matrix entries; measure the error reduction.

Challenge. Lyapunov equation solver; determinant via work statistics at scale.
Deliverable. Time-to-error curves, breakdown of time by stage.

L9 — Diffusion Order Finding and Factoring (Week 9)

Objective. Implement the diffusion analogue of Shor's algorithm and audit its resources.
Core tasks.

1. Compute $p_n(e) = \frac{1}{r}\sum_k \lambda_k^n$ from the character eigenvalues $\lambda_k = \frac{1}{2}\left(1 + \frac{1}{M+1}\sum_t \cos\frac{2\pi k 2^t}{r}\right)$, and also by direct sparse diffusion for small r.
2. Reproduce $N = 299$, $b = 3$ ($r = 33$; rounding correct from $n = 17$) and $N = 1{,}022{,}117$, $b = 576$ ($r = 5313$).
3. Implement the full factoring loop (random a, oddify, diffusion order finding, lift, gcd).
4. Collision method: factor $F_5 = 2^{32} + 1$ and 8,219,999; plot the running gcd.
5. Verify the identity $\sum_j \cos^n(2\pi j/m) = \frac{m}{2^n}\sum_k \binom{n}{(km+n)/2}$ numerically.
6. Resource table for 16- to 2048-bit N: vertices, diffusion steps n_0, analog precision bits.

Challenge. ngspice simulation of the 3-node RC ring for $N = 21$; add thermal noise and measure readout error.
Deliverable. Plots of $1/p_n(e)$, factorisation logs, resource table, one-page critical comparison with Shor.

L10 — Thermodynamic Natural Gradient Descent (Week 10)

Objective. Simulate TNGD and understand the SGD–NGD interpolation.
Core tasks.

1. Train a small CNN on MNIST; implement GGN–vector products with `jvp/vjp`.
2. Simulate $d\tilde{g} = -((G + \lambda I)\tilde{g} - \nabla\ell)\,dt + \mathcal{N}(0, 2\kappa_0 dt)$ for analog times $t \in \{0, 1, 5, 50\}\tau$.
3. Compare Adam, TNGD and TNGD-Adam with $(\beta_1, \beta_2) = (0, 0)$; study the delay t_d (momentum effect) and the noise level κ_0.
4. Log the residual $\|(G + \lambda I)\hat{x} - g\|/\|g\|$ against SDE steps.

Challenge. LoRA fine-tuning of a small language model with TNGD-Adam.
Deliverable. Loss against iterations and against estimated wall-clock; residual curves.

L11 — Thermodynamic K-FAC and Quantisation (Week 11)

Objective. Make second-order optimisation scalable.
Core tasks.

1. Implement K-FAC for an MLP or CNN (Kronecker factors A_ℓ, G_ℓ, EMA, damping).
2. Replace the inversions by simulated OU solves (linear-systems method).
3. Quantise inputs and outputs of the solver to 6, 8, 12 and 16 bits; reproduce the finding that output precision matters more.

4. Profile the fraction of step time spent in inversion as width n grows.

Challenge. Train a 74-unit non-linear Langevin computer (quartic on-site potential) as a classifier read out at a fixed time; measure accuracy against temperature.
Deliverable. Accuracy against bits, inversion share against n.

L12 — Diffusion Models and a DTM-style Denoising Chain (Week 12)

Objective. Connect diffusion models with sampling hardware.
Core tasks.

1. Train a DDPM on a 2D toy distribution and on binarised MNIST.
2. Build a DTM-style chain: each reverse step is a conditional RBM/EBM sampled with block Gibbs (THRML if available).
3. Plot sample quality against Gibbs sweeps per step and number of steps; project energy per sample with the fabric + I/O model.

Challenge. Precision schedule: fewer samples in high-noise steps; quantify the energy saved at equal quality.
Deliverable. Samples, quality–energy frontier.

L13 — Masked Diffusion LM versus Autoregressive LM (Week 13)

Objective. Compare the two language-model paradigms fairly.
Core tasks.

1. Character-level corpus; train a small causal AFT-conv LM and a bidirectional AFT denoiser (absorbing-state diffusion, cosine schedule) with equal parameters.
2. Implement the masked-diffusion ELBO so that both report nats/token (sanity check: a uniform predictor gives $\log V$).
3. Sweep the number of sampling steps $T \in \{8, 32, 128\}$; report quality and projected energy per token ($\propto T$).
4. Log GPU J/token with NVML for training and inference.

Challenge. Train both with TNGD-Adam and compare against AdamW at equal wall-clock and equal energy.
Deliverable. Bits/token table, quality against T, J/token table.

L14 — Z1T-ification: Sparse, Low-Precision, Sampled Layers (Week 14)

Objective. Turn a dense model into a TSU-mappable one and measure the price.
Core tasks.

1. Add fixed-connectivity masks ($c \in \{4, 16, 64, \text{dense}\}$) to all projections; DyT norms; tanh MLP.
2. Run a mini iso-FLOP sweep; plot loss against FLOPs per c and estimate the "sparsity tax".
3. Simulate dy4p encoding with Bernoulli streams and a sampled tanh-linear layer; plot accuracy or perplexity against samples S and effective bits.
4. Apply 4-bit quantisation-aware training.
5. Fill a TSU-mapping record (nodes, degree, colours, sweeps, I/O bits) and project J/token for Z1 + FPGA versus GPU (report system and fabric-only ratios).

Challenge. Design a disaggregation plan (what runs on TSU, GPU, FPGA) and estimate

latency per token.
Deliverable. Scaling plot, precision plot, "headline versus net" energy table.

L15 — A Mini Research-Agent Benchmark (Week 15)

Objective. Understand how LLM agents are trained and evaluated for thermodynamic ML research.
Core tasks.

1. Package five earlier labs as tasks with reference metrics.
2. Write an automatic execution score (0.15 for running, 0.85 for reproducing the metrics) and a rubric scored by an LLM judge; combine them as $r = 0.7\,\text{exec} + 0.3\,\text{rubric}$.
3. Evaluate one or two LLMs (API or open-weights) on the tasks; analyse failure modes and possible reward hacking.

Challenge. Port one lab to THRML or Torx, or run it on the Extropic simulator API if access is available.
Deliverable. Benchmark repository, results table, short ethics and reliability note.

9 Capstone Research Projects

Teams of 2–3 students choose one project (or propose their own). Each project must include a digital baseline, an energy model with evidence labels, and a reproducibility package.

ID	Project	Scope and expected results
P1	Thermodynamic diffusion LM with TNGD	AFT denoiser and AR baseline trained with AdamW, TNGD and TNGD-Adam; ELBO-based comparison; measured GPU J/token and projected TSU J/token; analog-time sweep.
P2	DTM for images in THRML	Chain of conditional EBMs on Fashion-MNIST-like data; quality against sweeps; energy projection against a small DDPM on GPU.
P3	Mini Z1T scaling law	Connectivity sweep with 4-bit weights; iso-FLOP curves; estimate of the sparsity tax and of the compute needed to match a dense baseline.
P4	Diffusion factoring hardware study	SPICE RC networks for small N; noise and precision analysis; engineering of the collision method; critical comparison with Shor and Pollard's rho.
P5	Thermodynamic K-FAC for a tiny ViT or GPT	Simulated OU solves per factor; quantisation; timing model; treatment of vocabulary-sized factors.
P6	Brownian token computer	Gillespie simulation of adders; first-passage statistics; TUR and cost of resetting; energy–time trade-offs.
P7	Non-equilibrium Langevin computer	Train for a readout time; optimise loading protocols to maximise η; clock-acceleration study.
P8	Heterogeneous inference planner	Tool that maps a model graph onto TSU + GPU + FPGA and estimates J/token and latency, validated on Z1T's published figures.

Milestones. Week 6: one-page proposal (question, baseline, metrics). Week 11: progress report with a working baseline. Week 15: final code and an IEEE-style report of 6–8 pages. Week 16: 15-minute talk plus 5 minutes of questions, and peer review of another team's report.

10 Assessment Plan

Component	Weight	When
Laboratory notebooks and reports (best 12 of 15)	35%	Weekly
Weekly concept quizzes (10 minutes, in Session A)	5%	Weekly
Journal club presentation and participation	10%	Rotating
Midterm exam (Units 0–5; derivations and short problems)	15%	Week 8
Capstone project: proposal 5%, progress 5%, report and code 15%, talk 10%	35%	Weeks 6, 11, 15, 16
Total	**100%**	

10.1 Laboratory rubric

Criterion	Points	Excellent level
Correctness	40	Results match theory or the reference within stated error bars; tests pass.
Analysis and plots	25	Clear figures with units; correct interpretation; limitations discussed.
Code quality and reproducibility	20	Seeds fixed, one-command rerun, readable functional JAX code.
Energy accounting and evidence labels	15	Explicit cost model; every number labelled [M]/[S]/[P]/[T]/[E].

10.2 Capstone rubric

Criterion	Points	Excellent level
Technical depth	25	Correct, non-trivial physics and ML; sound derivations.
Experimental rigour	25	Fair baselines, ablations, error bars, honest negative results.
Energy and hardware realism	20	Credible cost model; distinguishes system from fabric gains; identifies bottlenecks.
Originality	15	Goes beyond reproduction: a new ablation, idea or tool.
Communication	15	Clear report and talk; good answers to questions.

11 Video References

★ = core (watch before class); ☆ = optional or complementary. Every YouTube link was checked against its published title in October 2026. Some talks are advocacy or opinion (especially V6); watch them critically.

Code		Title — channel / speaker	Units	Link
V1	★	*From One to One Billion* (Torx, Thermalizers, Z1) — Extropic	5, 10, 11	https://www.youtube.com/watch?v=ITkoPau3k98
V2	★	*Thermodynamic AI and the Fluctuation Frontier* — Patrick Coles, Qiskit Seminar Series	0, 1, 6	https://www.youtube.com/watch?v=dd1jURhLR8Y
V3	★	*Thermodynamic computing for AI applications* — Patrick Coles, IQuS	5, 6	https://www.youtube.com/watch?v=zdmxeY8qdgU
V4	★	*Probabilistic Computing & p-bits for Optimization, ML and Quantum* — Kerem Çamsari, COINFLIPS Seminar	4, 5	https://www.youtube.com/watch?v=ais8rFqJPiM
V5	☆	*Probabilistic Computing with p-bits: Optimization, ML & Quantum Simulation* — Rutgers Efficient AI Seminar	5	https://www.youtube.com/watch?v=-z-cQxjCRV8
V6	☆	*Guillaume Verdon: Beff Jezos, E/acc Movement, Physics, Computation & AGI* — Lex Fridman Podcast #407 (context; opinionated)	0	https://www.youtube.com/watch?v=8fEEbKJoNbU
V7	☆	*The Most Misunderstood Concept in Physics* (entropy) — Veritasium	1	https://www.youtube.com/watch?v=DxL2HoqLbyA
V8	★	*From Stochastic Thermodynamics to Thermodynamic Inference* — Udo Seifert, BIRS workshop 20w5074 (2020)	3	https://www.birs.ca/events/2020/5-day-workshops/20w5074/videos/watch/202007270816-Seifert.html
V9	★	*Lecture 12.3 — Restricted Boltzmann Machines* (Neural Networks for Machine Learning) — Geoffrey Hinton	4	https://www.youtube.com/watch?v=JvF3gninXi8
V10	☆	*Spectral Graph Theory* boot-camp talks — Luca Trevisan, Simons Institute	0, 7	https://simons.berkeley.edu/talks/spectral-graph-theory
V11	★	*How Quantum Computers Break Encryption | Shor's Algorithm Explained* — minutephysics	7	https://www.youtube.com/watch?v=lvTqbM5Dq4Q
V12	★	*Diffusion and Score-Based Generative Models* — MIT CBMM	2, 9	https://www.youtube.com/watch?v=wMmqCMwuM2Q
V13	★	*Simple Diffusion Language Models* — Sasha Rush	9, 10	https://www.youtube.com/watch?v=WjAUX23vgfg
V14	☆	*Diffusion Language Models: The Next Big Shift in GenAI* — Jia-Bin Huang	9, 10	https://www.youtube.com/watch?v=8BTOoc0yDVA
V15	★	*Transformers, the tech behind LLMs* (Deep Learning Ch. 5) — 3Blue1Brown	10	https://www.youtube.com/watch?v=wjZofJX0v4M
V16	★	*Attention in transformers, step-by-step* (Deep Learning Ch. 6) — 3Blue1Brown	10	https://www.youtube.com/watch?v=eMlx5fFNoYc
V17	☆	*Let's build GPT: from scratch, in code, spelled out* — Andrej Karpathy	10 (L13)	https://www.youtube.com/watch?v=kCc8FmEb1nY
V18	☆	*Score-Based Generative Modeling through Stochastic Differential Equations* — Song et al., ICLR 2021 (SlidesLive)	9	https://slideslive.com/38954067

Complementary non-video resources. MIT 6.S184 *Introduction to Flow Matching and Diffusion Models* (slides and lecture notes): https://diffusion.csail.mit.edu/. The Extropic article *TSU 101: An Entirely New Type of Computing Hardware*: https://extropic.ai/writing/tsu-101-an-entirely-new-type-of-computing-hardware. Short animations (primitives, factor graphs, DTM denoising) are embedded in the *From One to One Billion* article [22]. Hinton's Nobel lecture *Boltzmann Machines* (8 December 2024, text): https://www.nobelprize.org/prizes/physics/2024/hinton/lecture/.

12 Journal Club Reading List

Each student presents one paper. Presentations must state the claim, the evidence label of each key number, the baseline used, and one experiment that could falsify the claim.

JC1. Coles et al., *Thermodynamic AI and the fluctuation frontier* [28].

JC2. Aifer et al., *Thermodynamic linear algebra* [30].

JC3. Melanson et al., *Thermodynamic computing system for AI applications* [25].

JC4. Donatella et al., *Thermodynamic natural gradient descent* [40].

JC5. Donatella et al., *Scalable thermodynamic second-order optimization* [41].

JC6. Whitelam, *The thermodynamic freedom of a thermodynamic computer* [42].

JC7. Whitelam and Casert, *Thermodynamic computing out of equilibrium* [43].

JC8. Utsumi et al., *Computation time and TUR of Brownian circuits* [26].

JC9. Raux, Goupil and Verley, *Thermodynamic circuits for chemical reaction networks* [27].

JC10. Cadavid et al., *Diffusion computation versus quantum computation* [33].

JC11. Jelinčič et al., *Probabilistic hardware architecture for diffusion-like models* (DTM) [48].

JC12. Extropic, *Z1T: sparse transformer-like models for probabilistic hardware* [23].

JC13. Extropic, *First sparks of thermodynamic recursive intelligence* [24].

JC14. Kaiser and Datta, *Probabilistic computing with p-bits* [18].

JC15. Sahoo et al., *Simple and effective masked diffusion language models* [47].

JC16. Frantar et al., *Scaling laws for sparsely-connected foundation models* [52].

13 Bibliography

[1] C. W. Gardiner, *Handbook of Stochastic Methods*, Springer (also H. Risken, *The Fokker–Planck Equation*, Springer, 1996).

[2] L. Peliti and S. Pigolotti, *Stochastic Thermodynamics: An Introduction*, Princeton University Press, 2021.

[3] D. A. Levin, Y. Peres and E. L. Wilmer, *Markov Chains and Mixing Times*, AMS, 2009.

[4] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.

[5] P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*, Springer, 1992.

[6] T. Conte et al., *Thermodynamic Computing* (CCC workshop report), 2019. https://arxiv.org/abs/1911.01968

[7] Quanta Magazine, *Thermodynamic Computers Go With the (Energy) Flow*, 15 July 2026. https://www.quantamagazine.org/thermodynamic-computers-go-with-the-energy-flow-20260715/

[8] R. Landauer, Irreversibility and heat generation in the computing process, *IBM J. Res. Dev.* 5, 183 (1961).

[9] C. H. Bennett, Notes on Landauer's principle, reversible computation and Maxwell's demon, 2003. https://arxiv.org/abs/physics/0210005

[10] D. H. Wolpert, The stochastic thermodynamics of computation, *J. Phys. A* 52, 193001 (2019).

[11] U. Seifert, Stochastic thermodynamics, fluctuation theorems and molecular machines, *Rep. Prog. Phys.* 75, 126001 (2012).

[12] C. Jarzynski, Nonequilibrium equality for free energy differences, *Phys. Rev. Lett.* 78, 2690 (1997).

[13] H. G. Katzgraber, Introduction to Monte Carlo methods, 2009. https://arxiv.org/abs/0905.1629

[14] D. H. Ackley, G. E. Hinton and T. J. Sejnowski, A learning algorithm for Boltzmann machines, *Cognitive Science* 9, 147 (1985).

[15] J. J. Hopfield, Neural networks and physical systems with emergent collective computational abilities, *PNAS* 79, 2554 (1982).

[16] G. E. Hinton, A practical guide to training restricted Boltzmann machines, 2010/2012. https://www.cs.toronto.edu/~hinton/absps/guideTR.pdf

[17] K. Y. Camsari, B. M. Sutton and S. Datta, p-bits for probabilistic spin logic, *Appl. Phys. Rev.* 6, 011305 (2019).

[18] J. Kaiser and S. Datta, Probabilistic computing with p-bits, *Appl. Phys. Lett.* 119, 150503 (2021). https://arxiv.org/abs/2108.09836

[19] N. A. Aadit et al., Massively parallel probabilistic computing with sparse Ising machines, *Nat. Electron.* 5, 460 (2022).

[20] A full-stack view of probabilistic computing with p-bits, 2023. https://arxiv.org/abs/2302.06457

[21] Extropic, *Thermodynamic Computing From Zero to One*, 29 October 2025. https://extropic.ai/writing

[22] Extropic, *From One to One Billion: Torx, Thermalizers, and Z1*, 2026. https://extropic.ai/writing/from-one-to-one-billion

[23] Extropic (G. Verdon et al.), *Z1T: Sparse Transformer-Like Models for Probabilistic Hardware*, 4 September 2026. https://extropic.ai/writing

[24] Extropic and Prime Intellect, *First Sparks of Thermodynamic Recursive Intelligence*, 1 October 2026. https://extropic.ai/writing

[25] D. Melanson et al., Thermodynamic computing system for AI applications, *Nat. Commun.* 16, 3757 (2025). https://www.nature.com/articles/s41467-025-59011-x

[26] Y. Utsumi et al., Computation time and thermodynamic uncertainty relation of Brownian circuits, 2022. https://arxiv.org/abs/2205.10735

[27] P. Raux, C. Goupil and G. Verley, Thermodynamic circuits: modeling chemical reaction networks with nonequilibrium conductance matrices, 2025. https://arxiv.org/abs/2412.15028

[28] P. J. Coles et al., Thermodynamic AI and the fluctuation frontier, 2023. https://arxiv.org/abs/2302.06584

[29] M. Aifer et al., Solving the compute crisis with physics-based ASICs, 2025. https://arxiv.org/abs/2507.10463

[30] M. Aifer et al., Thermodynamic linear algebra, *npj Unconv. Comput.* 1, 13 (2024). https://doi.org/10.1038/s44335-024-00014-0

[31] M. Aifer et al., Error mitigation for thermodynamic computing, 2024. https://arxiv.org/abs/2401.16231

[32] C. A. Cadavid et al., An integer factorization algorithm which uses diffusion as a computational engine, 2023. https://arxiv.org/abs/2104.11616

[33] C. A. Cadavid et al., Diffusion computation versus quantum computation: a comparative model for order finding and factoring, 2026. https://arxiv.org/abs/2601.02518

[34] C. A. Cadavid et al., On an approach for evaluating certain trigonometric character sums using the discrete time heat kernel, 2022. https://arxiv.org/abs/2201.07878

[35] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, *SIAM J. Comput.* 26, 1484 (1997).

[36] S.-I. Amari, Natural gradient works efficiently in learning, *Neural Computation* 10, 251 (1998).

[37] J. Martens, New insights and perspectives on the natural gradient method, *JMLR* 21 (2020). https://arxiv.org/abs/1412.1193

[38] J. Martens and R. Grosse, Optimizing neural networks with Kronecker-factored approximate curvature, ICML 2015. https://arxiv.org/abs/1503.05671

[39] R. Grosse, CSC2541 lecture notes, *Second-order optimization*, 2021. https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2021/readings/L04_second_order.pdf

[40] K. Donatella et al., Thermodynamic natural gradient descent, 2024. https://arxiv.org/abs/2405.13817

[41] K. Donatella et al., Scalable thermodynamic second-order optimization, 2025. https://arxiv.org/abs/2502.08603

[42] S. Whitelam, The thermodynamic freedom of a thermodynamic computer, 2026. arXiv:2608.27938.

[43] S. Whitelam and C. Casert, Thermodynamic computing out of equilibrium, 2024. https://arxiv.org/abs/2412.17183

[44] J. Ho, A. Jain and P. Abbeel, Denoising diffusion probabilistic models, NeurIPS 2020. https://arxiv.org/abs/2006.11239

[45] Y. Song et al., Score-based generative modeling through stochastic differential equations, ICLR 2021. https://arxiv.org/abs/2011.13456

[46] P. Holderrieth and E. Erives, An introduction to flow matching and diffusion models (MIT 6.S184 notes), 2025. https://arxiv.org/abs/2506.02070

[47] S. S. Sahoo et al., Simple and effective masked diffusion language models, NeurIPS 2024. https://arxiv.org/abs/2406.07524

[48] A. Jelinčič et al., An efficient probabilistic hardware architecture for diffusion-like models, *npj Unconv. Comput.* 3, 30 (2026). https://doi.org/10.1038/s44335-026-00075-3

[49] A. Vaswani et al., Attention is all you need, NeurIPS 2017.

[50] S. Zhai et al., An attention free transformer, 2021. https://arxiv.org/abs/2105.14103

[51] J. Zhu, X. Chen, K. He, Y. LeCun and Z. Liu, Transformers without normalization, CVPR 2025.

[52] E. Frantar et al., Scaling laws for sparsely-connected foundation models, 2023. https://arxiv.org/abs/2309.08520

[53] J. Hoffmann et al., An empirical analysis of compute-optimal large language model training, NeurIPS 2022.

[54] G. Verdon et al., A framework for stochastic differentiable programming (Torx), 2026. https://arxiv.org/abs/2608.01612

[55] M. Amico et al., Thermalizing stochastic programs (Thermalizers), 2026. https://arxiv.org/abs/2608.01615

[56] O. Lockwood et al., A blueprint for equilibrium-based differentiable continuous-variable thermodynamic computing, 2026. https://arxiv.org/abs/2607.16183

[57] Z. Shao et al., DeepSeekMath (introduces GRPO), 2024. https://arxiv.org/abs/2402.03300

A Prerequisite Self-Check (Week 1)

1. Compute the mean and covariance of $x = Ay + b$ when $y \sim \mathcal{N}(0, I)$.
2. Show that an SPD matrix has positive real eigenvalues; define its condition number.
3. State detailed balance and show that it implies stationarity.
4. Write $(B \otimes C)^{-1}$ and $(B \otimes C)\,\mathrm{vec}(X)$ in terms of B, C, X.
5. Compute the entropy of a Bernoulli(p) variable and of a d-dimensional Gaussian.
6. Compute $\sum_{j=0}^{r-1} e^{2\pi i j k / r}$ for integer k.

7. Write a JAX function that draws 10^6 standard normal samples reproducibly and computes their mean with `jit`.
8. Explain the difference between a gradient and a Jacobian–vector product.

B Formula Sheet

Boltzmann distribution	$p(x) = e^{-\beta E(x)}/Z, \quad \beta = 1/k_B T$
Landauer bound	$k_B T \ln 2 \approx 2.87 \times 10^{-21}$ J at 300 K
Overdamped Langevin	$\dot{x} = \mu f(x) + \sqrt{2\mu k_B T}\,\xi(t)$
OU linear solver	$dx = -(Ax - b)dt + \sqrt{2\beta^{-1}}dW \Rightarrow x_\infty \sim \mathcal{N}(A^{-1}b, \beta^{-1}A^{-1})$
Jarzynski	$\langle e^{-\beta W}\rangle = e^{-\beta \Delta F}$
Speed limit, efficiency	$\tau \geq W_2^2/(\mu k_B T \Sigma), \quad \eta = W_2^2/(\mu \tau k_B T \Sigma) \leq 1$
TUR (first passage)	$\langle \tau \rangle / \sqrt{\mathrm{Var}\,\tau} \leq \sqrt{\Sigma_{\mathrm{bi}}/2 + A_{\mathrm{uni}}}$
pbit conditional	$P(s_i{=}{+}1) = \sigma(2\beta u_i), \quad \mathbb{E}[s_i] = \tanh(\beta u_i), \quad u_i = h_i + \sum_j J_{ij} s_j$
Boltzmann learning	$\Delta J_{ij} \propto \langle s_i s_j \rangle_{\mathrm{data}} - \langle s_i s_j \rangle_{\mathrm{model}}$
Heat-kernel bound	$\lvert p_n(x) - 1/k \rvert \leq \lambda_1^n$
Diffusion order finding	$p_n(e) = \frac{1}{r}\sum_k \lambda_k^n, \quad n_0 = 4(M+1)(\log_2 N + 2), \quad r = \mathrm{round}(1/p_{n_0}(e))$
TNGD	$d\tilde{g} = -((G + \lambda I)\tilde{g} - \nabla \ell)dt + \mathcal{N}(0, 2\kappa_0 dt)$
K-FAC update	$U_\ell = G_\ell^{-1}(D\Theta_\ell)A_{\ell-1}^{-1}$
Effective bits	$b_{\mathrm{eff}} \approx b_1 + \frac{1}{2}\log_2 S$ (S = samples averaged)
Energy model	$E = n_{\mathrm{nodes}}\, n_{\mathrm{sweeps}}\, e_{\mathrm{update}} + b_{\mathrm{I/O}}\, e_{\mathrm{bit}} + E_{\mathrm{digital}}$
Diffusion LM cost	$E_{\mathrm{token}} \approx T \cdot e_{\mathrm{pos}}$ (AR with cache: $\approx e_{\mathrm{pos}}$)

C Sample Exam Questions

1. Derive the stationary distribution of the OU process and explain how its first two moments solve $Ax = b$ and A^{-1}. Why does the time to solution grow with κ?
2. A chip has 269,568 pbits, runs at 50 MHz and consumes under 1 W. Bound the energy per pbit update and compare it with $k_B T \ln 2$. Why is this comparison not the one that decides the system-level efficiency?
3. Show that the conditional mean of an Ising spin is $\tanh(\beta u_i)$ and explain how averaging produces a tanh-linear layer. How many samples give about 6 effective bits?
4. Explain the clock-acceleration procedure. Which quantities change and which do not?
5. Compare Shor's algorithm and diffusion order finding: mechanism, step count, physical resources and precision. Why does the latter not threaten RSA-2048?
6. A diffusion LM uses $T = 200$ denoising steps. Using $E_{\mathrm{token}} \approx T \cdot e_{\mathrm{pos}}$, compare it with autoregressive decoding on the same thermodynamic hardware. Which levers reduce the gap?
7. Z1T reports 294.52 nJ/token, of which 285.78 nJ is FPGA work. Compute the FPGA share and discuss what a next-generation chip must change to approach 1000× gains.
8. Why is full-model TNGD infeasible for a 22M-parameter language model, while thermodynamic K-FAC is feasible? Estimate the size of the largest Kronecker factor for a model with width 256 and MLP width 1024.

D Lab Environment Setup

```
python -m venv .venv && source .venv/bin/activate
pip install "jax[cuda12]" equinox optax numpy scipy matplotlib networkx sympy
pip install nvidia-ml-py            # GPU power -> J/token, J/sample
# THRML: follow the install instructions at https://docs.thrml.ai
# Torx:  follow the install instructions at https://docs.torx.ai
# ngspice: install from your operating system's package manager
```